

Système d'exploitation

Système d'exploitation - Utilisation

Mickael Rouvier

CERI – Avignon Université
`mickael.rouvier@univ-avignon.fr`

February 17, 2026

Section 1

Présentation sommaire de l'UCE

Plan et objectifs du cours

- UE: Système d'exploitation
 - **Système d'exploitation - Utilisation (L1)**
 - Système d'exploitation - Administration (L2)
 - Système d'exploitation - Programmation (L3)
- Objectifs du cours
 - Découverte et prise en main de l'environnement GNU/Linux
 - Connaissance du fonctionnement GNU/Linux
- Plan du cours
 - **Cours 1**
 - Historique du SE
 - Terminal
 - Gestion des fichiers
 - **Cours 2**
 - Historique des commandes
 - Manipulation des fichiers
 - **Cours 3**
 - Redirection des flux
 - Archive compressé
 - **Cours 4**
 - Programmation en bash
 - **Cours 5**
 - Gestion de la mémoire et des processus
 - Noyau, distribution et processus de démarrage

- **Attention** : il y a des choses que je ne dis qu'en cours, donc venez et soyez à l'heure
- Notes
 - TP Final (Coefficient 0.5)
 - Examen : Coefficient (0.5)

Section 2

Introduction Système d'Exploitation (SE)

Introduction au Système d'Exploitation (SE)

- **SE les plus connus**

- Windows (dernière version : Windows 11)
- GNU/Linux (distributions de bureau : Debian, Ubuntu, Fedora, etc.)
- macOS (dernière version : macOS Ventura)
- Android et iOS (smartphones et tablettes)

- **Qu'est-ce qu'un SE ? À quoi sert un SE ?**

- Historique : Quand les SE sont-ils apparus ?
- Fonctionnalités principales : Quel est le rôle d'un SE ?
- Services fournis : Quels sont les services rendus par un SE ?



Historique – Première génération : Tubes à vide (1945)

- **Premier ordinateur : ENIAC**

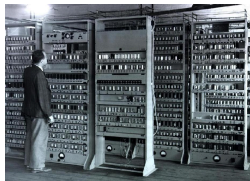
- Les premières machines à calculer utilisaient des tubes à vide
- Ces machines énormes occupaient des salles entières et étaient moins rapides qu'une calculatrice moderne
- Applications : tables de tir, déchiffrement de message pendant la Seconde Guerre mondiale ¹

- **Pas besoin de système d'exploitation**

- Toute la préparation se faisait manuellement (câblage des branchements)
- Le programmeur interagissait directement avec la machine, nécessitant de nombreuses manipulations
- Rôle unique : concepteur, constructeur, programmeur et opérateur étaient souvent la même personne

- **Limites**

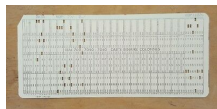
- Utilisation trop complexe
- Le temps de préparation était très long



¹https://fr.wikipedia.org/wiki/Imitation_Game

Historique – Deuxième génération : Traitement par lots (1956)

- **Apparition des ordinateurs programmables**
 - Automatisation d'une suite de commandes grâce aux cartes perforées
 - Enchaînement automatique des commandes : le **traitement par lots**
 - Séparation des rôles : le programmeur n'est pas l'opérateur
- **Problèmes : Programmes bouclent et accès mémoires**
 - Certains programmes bouclent indéfiniment et bloquent la machine
 - Accès direct à la mémoire pouvant détruire d'autres programmes
 - Temps machine très coûteux et mal utilisé
- **Solution : introduction des premiers systèmes d'exploitation**
 - Contrôler l'exécution des programmes
 - Gérer la mémoire pour éviter les conflits
 - Organiser les entrées/sorties
- **Objectif** : mieux utiliser des machines très coûteuses et éviter les erreurs



Historique – Troisième génération : Temps partagé (1961)

- **Problème : Accès aux données sur disque dur trop long**
 - Les opérations de lecture/écriture sur disque étaient lentes
 - **Solution** : Utilisation de la mémoire vive pour réduire les temps d'accès
- **Problème : Besoin d'accès multi-utilisateurs**
 - Introduction du **temps partagé** permettant à plusieurs utilisateurs d'accéder simultanément au système
 - **Exemple** : Multics², CTSS (Compatible Time-Sharing System)
- **Nouveaux enjeux**
 - Multi-utilisateur
 - Multi-tâches
 - Début des réseaux
 - Sécurité et gestion des accès
- **Objectif** : utiliser l'ordinateur plus efficacement

²<https://fr.wikipedia.org/wiki/Multics>

Historique – Quatrième génération : Apparition de la micro-informatique (1980)

- **Apparition des ordinateurs individuels (PC)**
- **Problème : Diversité des matériels (cartes son, CD-ROM, réseau...)**
 - Nécessité de gérer une grande variété de périphériques
 - Introduction des pilotes de périphériques
- **Problème : Besoin d'une interface utilisateur conviviale**
 - Développement des interfaces graphiques (GUI) comme X11, Xerox Alto
 - Facilitation de l'interaction utilisateur avec le système
 - **Exemple** : Interface graphique de Windows 3.0
- **Problème : Diversification des supports de stockage**
 - Adaptation des systèmes d'exploitation aux nouveaux supports (disques durs, SSD, USB, etc.)
- **Nouveaux enjeux**
 - Réseaux et Internet
 - Sécurité informatique
 - Mobilité (smartphones)
 - Cloud et virtualisation

Historique - Résumé

- **Première génération** : Accès direct, sans système d'exploitation
- **Deuxième génération** : Traitement par lots
- **Troisième génération** : Temps partagé
- **Quatrième génération** : Micro-informatique et ordinateurs individuels

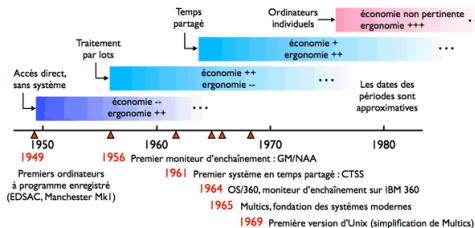
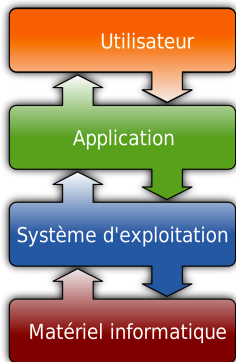


Figure 1: Évolution des générations des systèmes d'exploitation

Quel est le rôle d'un SE



- **Intermédiaire entre le matériel et les applications**

- Fait le lien entre les ressources matérielles d'un ordinateur et les applications de l'utilisateur (par exemple, fournit une interface commune pour écrire sur un fichier situé sur un disque, une clé USB ou un réseau...)

- **Gestion des ressources et des accès**

- Assure le démarrage et gère l'accès aux ressources communes entre plusieurs utilisateurs ou programmes (par exemple, la mémoire, le disque, l'accès réseau...)

Quels sont les services rendus par un SE ?

• Gestion des fichiers

- Organisation des fichiers en arborescence
- Gestion physique sur les supports de stockage (disque dur, clé USB, etc.)
- **Exemple** : Système de fichiers NTFS, ext4, APFS

• Gestion de la mémoire

- Allocation de la mémoire à chaque application
- Gestion de la mémoire entre plusieurs applications de manière sécurisée
- **Exemple** : Pagination, segmentation, mémoire virtuelle

• Gestion des entrées/sorties (E/S)

- Contrôle des périphériques (cartes réseau, cartes son, imprimantes...)
- **Exemple** : Pilotes de périphériques qui permettent au système de communiquer avec une imprimante

• Gestion des applications

- Ordonnancement des processus (algorithmes de scheduling)
- Création, suspension, reprise et terminaison des processus
- Gestion des threads
- **Exemple** : Planificateur de tâches qui attribue du temps processeur aux applications en fonction de priorités

• Gestion d'une interface utilisateur

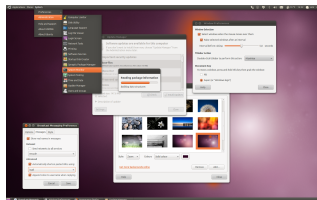
- Interface en ligne de commande (CLI) pour les interactions textuelles
- Interface graphique (GUI) via des gestionnaires de fenêtres (window manager, tiling manager, etc.)
- **Exemple** : Shell Bash pour la CLI, GNOME ou KDE pour la GUI sous Linux

Section 3

Terminal

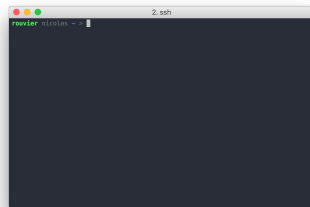
Comment interagir avec le SE via l'interface graphique ?

- **En général l'interaction se fait via des fenêtres, icônes, menus et boutons**
 - Interface intuitive pour les débutants
- **Limites de l'interface graphique**
 - **Nombreux clics nécessaires** : lent pour les tâches répétitives
 - **Peu reproductible** : difficile de répéter exactement une suite d'actions
 - **Automatisation difficile** : compliqué d'automatiser des clics souris
 - **Accès limité aux fonctions avancées** : certaines options ne sont pas visibles



Comment interagir avec le SE via le Terminal ?

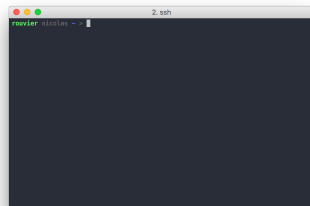
- **Terminal** : Un écran noir qui attend une commande
- **Fonctionnement du Terminal**
 - Lit et interprète les commandes saisies par l'utilisateur
 - Interagit avec le SE pour exécuter les tâches demandées
 - Attend la prochaine commande une fois la précédente terminée
- **Avantages du Terminal**
 - **Rapidité** : Certaines actions sont beaucoup plus rapides à écrire sur le terminal
 - **Exhaustivité** : Accès à toutes les informations contrairement à l'interface graphique qui n'affiche pas toujours toutes les informations
 - **Convivialité** : Donner une commande est parfois plus simple que d'expliquer là où il faut cliquer



Un Peu de Vocabulaire

- **Terminal** : Un écran noir en attente d'une instruction, d'une commande
- **Interpréteur de Commande** (ou **Shell**) :
 - Traite les commandes saisies, les interprète et exécute les tâches associées.
 - Exemples : Bash, Zsh, PowerShell.
- **Prompt** :

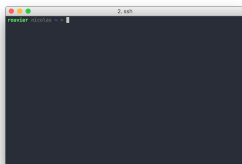
invite/> ■



Différence entre Terminal et Shell

• Terminal

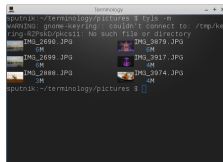
- Interface permettant de saisir des commandes et d'afficher les résultats.
- Exemples : GNOME Terminal, Konsole, Windows Terminal.



iTerm2



Cool Retro Term



Terminology

• Shell

- Programme qui interprète les commandes saisies dans le terminal.
- Exemples :
 - **Bash** (Bourne Again Shell) : Shell par défaut dans de nombreuses distributions Linux comme Ubuntu.
 - **Zsh** (Z Shell) : Offre des fonctionnalités avancées comme la complétion automatique et la correction orthographique.
 - **Csh** (C-Shell) : Utilisé pour sa syntaxe inspirée du langage C.

Le Terminal est-il dépassé ?

Absolument pas. Il est au cœur de l'informatique moderne.

Pourquoi le terminal reste indispensable :

- Utilisé par tous les développeurs (Git, compilation, Python, C/C++, etc.)
- Indispensable pour les serveurs Linux et le cloud
- Administration à distance via SSH
- Automatisation des tâches avec des scripts
- Plus rapide et plus puissant que l'interface graphique
- Essentiel en cybersécurité, IA et DevOps

Maîtriser le terminal = compétence clé en informatique

Le Terminal est-il dépassé ?

- **Le Terminal est-il dépassé ?** La question ne doit même pas se poser



- **Inconvénient :**

- **Courbe d'apprentissage élevée :** Nécessite la mémorisation des commandes et de leur syntaxe
- **Interface moins intuitive :** Moins accessible pour les utilisateurs non techniques ou débutants
- **Feedback visuel limité :** Absence de représentations graphiques immédiates des actions effectuées

- **Avantage :**

- **Efficacité et rapidité :** Exécution rapide des commandes sans surcharge graphique
- **Contrôle précis :** Accès aux fonctionnalités et options avancées des outils
- **Automatisation et scripts :** Possibilité de créer des scripts pour automatiser des tâches répétitives
- **Gestion à distance :** Facilité de gestion des serveurs et systèmes via SSH ou d'autres protocoles

- **Le Terminal n'est pas prêt de disparaître (ex. nouveau terminal de Windows 10).**

Command Line Argument – Partie 1

- **Définition d'une commande**

- Une **commande** est une instruction donnée à l'interpréteur de commandes (shell) pour exécuter une tâche spécifique.
- Elle permet d'interagir directement avec le système d'exploitation pour effectuer des opérations comme la gestion des fichiers, la configuration du système, l'exécution de programmes, etc.

- **Structure d'une commande**

- Une commande peut être suivie d'options et d'arguments
- Syntaxe générale :

```
invite/> command [-option(s)] [argument(s)]
```

- **Commande** : nom du programme (ex : ls, cp, mkdir)
- **Options** : modifient le comportement (-l, -a...)
- **Arguments** : Spécifient les données sur lesquelles la commande opère (fichiers ou dossiers concernés)

Command Line Argument – Partie 2

- **Convention POSIX :**

- Une option commence par un tiret suivi d'un caractère alphanumérique (ex : **-o**)
- Une option peut nécessiter un argument immédiatement après (ex : **-o argument** ou **-oargument**)
- Les options sans arguments peuvent être groupées après un tiret (ex : **-lst** équivaut à **-l -s -t**)
- Les options peuvent apparaître dans n'importe quel ordre (ex : **-lst** équivaut à **-lts**)
- Les options peuvent être répétées (ex : **-l -s -t -s**)

- **Options Longues avec GNU**

- Commencent par deux tirets suivis de mots ou expressions (ex : **-long-option**)
- Possibilité de spécifier un argument avec un signe égal (ex : **-long-option=argument**)

- **Différences entre Unix et BSD**

- Les options dans les programmes BSD peuvent ne pas commencer par un tiret
- Exemple : **tar zxvf file.tar.gz** vs **tar -zxvf file.tar.gz**

Command Line Argument – Partie 3

Gestion des arguments dans un programme

```
1 #!/bin/python
2 import sys
3
4 print sys.argv[0]
5 print sys.argv[1]
6 print sys.argv[2]
```

Listing 1: "Exemple de script en python : main.py"

```
1 #include <iostream>
2
3 int main(int argc, char** argv) {
4     cout<< argv[0] <<endl;
5     cout<< argv[1] <<endl;
6     cout<< argv[2] <<endl;
7
8     return 0;
9 }
```

Listing 2: "Exemple de script en python : main.cpp"

```
invite/> ./main.py aaa bbb
main.py
aa
bb
```

```
invite/> ./main aaa bbb
main
aa
bb
```

Obtenir de l'aide sur une commande - Partie 1

- **La commande `man`**

- Permet d'obtenir la documentation des commandes et programmes du système
- Syntaxe :

```
man nom_de_commande
```

- **Sections du manuel**

- Les pages du manuel sont divisées en plusieurs sections :
 - 1 Commandes utilisateur (Programmes exécutables)
 - 2 Appels système (Fonctions du noyau)
 - 3 Appels de bibliothèque (Fonctions des bibliothèques)
 - 4 Fichiers spéciaux (ex : `/dev`)
 - 5 Formats de fichiers et conventions (ex : `/etc/passwd`)
 - 6 Jeux
 - 7 Divers
 - 8 Commandes de gestion du système (généralement réservées au superutilisateur)
 - 9 Interfaces du noyau Linux

Obtenir de l'aide sur une commande - Partie 3

- **Documentation des Fonctions du Langage C**

- Utiliser `man` pour obtenir de l'aide sur les fonctions C standard
- Exemple :

```
invite/> man strcat
```

```
STRCAT(3)          BSD Library Functions Manual          STRCAT(3)

NAME
  strcat, strcat -- concatenate strings

LIBRARY
  Standard C Library (libc, -lc)

SYNOPSIS
  #include <string.h>

  char *
  strcat(char *restrict s1, const char *restrict s2);

  char *
  strncpy(char *restrict s1, const char *restrict s2, size_t n);

DESCRIPTION
  The strcat() and strncpy() functions append a copy of the null-terminated string s2 to the end of the
  null-terminated string s1, then add a terminating '\0'. The string s1 must have sufficient space to
  hold the result.

  The strncpy() function appends not more than n characters from s2, and then adds a terminating '\0'.
  The source and destination strings should not overlap, as the behavior is undefined.

RETURN VALUES
  The strcat() and strncpy() functions return the pointer s1.

SECURITY CONSIDERATIONS
  The strcat() function is easily misused in a manner which enables malicious users to arbitrarily
  change a running program's functionality through a buffer overflow attack. (See the FSA.)

  Avoid using strcat(). Instead, use strncpy() or strncpy() and ensure that no more characters are
  copied to the destination buffer than it can hold.
```

Section 4

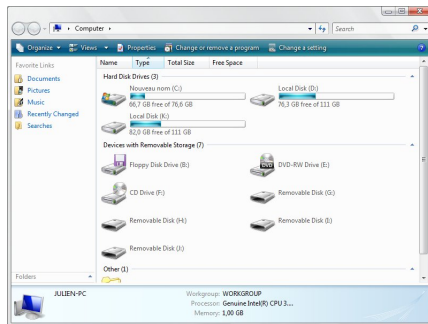
Système de Gestion de Fichiers

- **Définition** : Le système de gestion de fichiers est une composante essentielle d'un système d'exploitation. Il est responsable de la gestion du stockage, de l'organisation et de la manipulation des fichiers sur divers supports de stockage tels que les partitions, disques durs, CD, disquettes...
- **Types de Systèmes de Gestion de Fichiers** :
 - FAT, FAT32, NTFS (Windows)
 - Ext3, Ext4, Btrfs (Linux)
 - HFS, HFS+ (macOS)
- **Caractéristiques Clés** :
 - Taille maximale d'un fichier
 - Taille maximale d'une partition
 - Gestion des droits d'accès aux fichiers et répertoires
 - Journalisation pour assurer l'intégrité des données
 - Compression des données
 - Chiffrement des données pour la sécurité
 - Prise d'instantanés (snapshots) pour la sauvegarde et la restauration
 - Organisation hiérarchique des fichiers et dossiers

Organisation des Fichiers et Dossiers – Windows

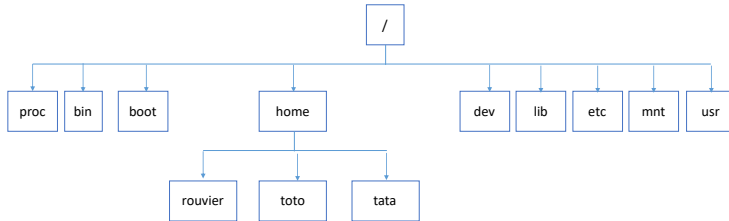
- **Modèle Hiérarchique Multi-Racines**

- Chaque disque ou partition est représenté par une lettre (C:, D:, E:, etc.)
- Structure hiérarchique indépendante pour chaque racine



• Modèle Hiérarchique Unique

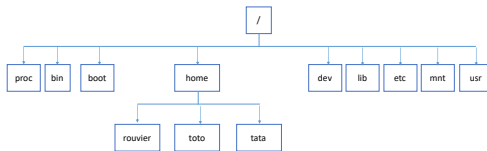
- Modèle hiérarchique en arbre inversé
- Un seul point de racine noté /
- Tous les périphériques et systèmes de fichiers sont montés sous cette racine



Arborescence des Répertoires - Détails sous Linux

• Principaux Répertoires :

- **/bin** : Contient les applications (exécutables) accessibles à tous les utilisateurs.
- **/boot** : Fichiers nécessaires au démarrage de Linux.
- **/dev** : Fichiers de périphériques (disques, terminaux, etc.).
- **/etc** : Fichiers de configuration du système.
- **/home** : Répertoires personnels des utilisateurs.
- **/mnt** : Points de montage pour les périphériques amovibles.
- **/opt** : Applications tierces et options supplémentaires.
- **/root** : Répertoire personnel de l'utilisateur root.
- **/usr** : Programmes et fichiers utilisés par les utilisateurs.
- **/var** : Fichiers variables (logs, caches, etc.).



Terminologie des répertoires spéciaux

- **Répertoire d'Accueil (home)**

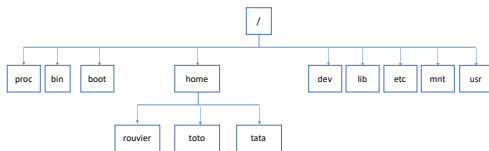
- Répertoire personnel de chaque utilisateur
- Symbolisé par ~ (tilde)
- Exemple : `/home/utilisateur`

- **Répertoire Courant (Working Directory)**

- Répertoire où l'on se trouve actuellement
- Symbolisé par . (un seul point)

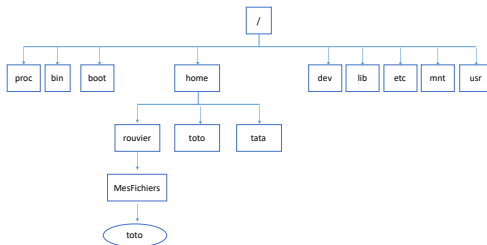
- **Répertoire Parent**

- Répertoire au-dessus du répertoire courant dans l'arborescence
- Symbolisé par .. (deux points)



Chemins d'Accès aux Fichiers

- **Chemin absolu** : Commence par la racine /
 - Exemple : `/home/rouvier/MesFichiers/toto`
- **Chemin relatif au répertoire d'accueil** :
 - Commence par `~/`
 - Exemple : `~/MesFichiers/toto`
- **Chemin relatif au répertoire courant** :
 - Commence par `./` ou directement par le nom du fichier/répertoire
 - Exemples : `./toto` ou `toto`



- **Les commandes sur les répertoires :**

- **cd** : **c**hange **d**irectory
- **mkdir** : **m**ake **d**irectory
- **rmdir** : **r**emove **d**irectory
- **pwd** : **p**rint **w**orking **d**irectory

Changer de répertoire avec cd

- **La commande cd permet de naviguer dans l'arborescence des répertoires**

- **Exemple :** Naviguer vers un répertoire

```
invite/> cd /etc
```

```
invite/> cd /home/rouvier/
```

- **Exemple :** Naviguer vers un répertoire supérieur (parent)

```
invite/> cd ..
```

```
invite/> cd ../dir1/dir2
```

- **Exemple :** Naviguer vers le répertoire d'accueil

```
invite/> cd ~
```

```
invite/> cd ~/dir1
```

Créer des répertoires avec `mkdir`

- La commande `mkdir` permet de créer un ou plusieurs répertoires

- **Exemple :** créer un répertoire

```
invite/> mkdir nom_repertoire
```

```
invite/> mkdir /etc/nom_repertoire
```

```
invite/> mkdir ../nom_repertoire
```

- **Exemple :** créer plusieurs répertoires successifs avec `-p`

```
invite/> mkdir -p repertoire1/repertoire2
```

- **Exemple :** afficher un message pour chaque création avec `-v`

```
invite/> mkdir -v repertoire1  
mkdir: création du répertoire 'repertoire1'
```


Supprimer des Répertoires avec `rmdir`

- La commande `rmdir` permet de supprimer des répertoires vides.

- **Exemple :** Supprimer un répertoire

```
invite/> mkdir nom_repertoire  
invite/> rmdir nom_repertoire
```

- **Exemple :** Supprimer un répertoire non vide (**attention** : le répertoire à supprimer doit impérativement être vide)

```
invite/> rmdir /etc  
rmdir: échec de suppression de « /etc »: Le dossier n'est pas vide
```

- **Exemple :** Supprimer un répertoire non vide

```
invite/> rm -rf nom_repertoire
```

Afficher le Répertoire Courant avec `pwd`

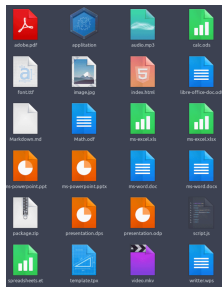
- La commande `pwd` affiche le chemin complet du répertoire dans lequel vous vous trouvez actuellement
 - **Exemple :** afficher le chemin complet du répertoire courant

```
invite/> pwd
/home/root
invite/> cd ..
invite/> pwd
/home
```

Identification des fichiers via l'interface graphique

- **Avec un Système de Fenêtrage / Icônes**

- Les fichiers sont représentés par des icônes selon leur type
- Facilite la reconnaissance visuelle des fichiers



Identification des fichiers via le Terminal

- **Nom d'un fichier** : généralement composé : préfixe.suffixe (ou encore appelé base.extension)
 - **Base/Préfixe** : Nom du fichier (ex : `document`)
 - **Extension/Suffixe** : Indique le type de fichier (ex : `.txt`, `.sh`, `.c`)
- **Exemple d'extension de fichiers** :
 - `.txt` : Fichier texte
 - `.sh` : Script shell
 - `.c`, `.cpp` : Programme en C/C++
 - Pas d'extension : Programme exécutable
- **Exemple de fichiers** :

```
Archive.zip  
test.dic  
chess.py  
main_EN.tex  
main_EN.pdf
```

- **Les commandes sur les fichiers :**

- **ls** : list
- **cp** : copy
- **mv** : move
- **rm** : remove
- **ln** : link

Lister les fichiers avec `ls`

- La commande `ls` permet de lister les fichiers et les répertoires dans un répertoire donné

- **Exemple :** Lister les fichiers dans le répertoire courant

```
invite/> ls
```

- **Exemple :** Lister tous les fichiers, y compris les fichiers cachés

```
invite/> ls -a
```

- **Exemple :** Afficher les détails des fichiers dans un format long

```
invite/> ls -l
```

- **Exemple :** Trier les fichiers par taille

```
invite/> ls -S
```

- **Exemple :** Combiner plusieurs options (par exemple, afficher en format long et inclure les fichiers cachés)

```
invite/> ls -la
```

Lister les fichiers avec ls

- La commande **ls** permet de lister les fichiers et les répertoires dans un répertoire donné
 - **Exemple** : Afficher les détails des fichiers dans un format long

```
invite/> ls -l
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 Archive.zip
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 Archive_cnn.zip
drw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 Chess-Engine
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 test.dic
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 toto.py
```

Copier des fichiers avec cp

- **La commande cp permet de copier des fichiers ou des répertoires**

- **Exemple :** Copier un fichier vers un autre emplacement

```
invite/> cp fichier1.txt /home/utilisateur/Documents/
```

```
invite/> cp /etc/config.conf ./backup_config.conf
```

- **Exemple :** Copier plusieurs fichiers vers un répertoire

```
invite/> cp fichier1.txt fichier2.txt /home/utilisateur/Documents/
```

- **Exemple :** Copier un répertoire de manière récursive

```
invite/> cp -r /home/utilisateur/dir1 /home/utilisateur/backup_dir1
```

- **Exemple :** Copier un fichier en conservant les attributs

```
invite/> cp -p fichier1.txt /home/utilisateur/Documents/
```


Déplacer ou renommer des fichiers avec `mv`

- **La commande `mv` permet de déplacer ou renommer des fichiers et des répertoires**

- **Exemple :** Déplacer un fichier vers un autre répertoire

```
invite/> mv fichier.txt /home/utilisateur/Documents/
```

- **Exemple :** Renommer un fichier

```
invite/> mv ancien_nom.txt nouveau_nom.txt
```

- **Exemple :** Déplacer et renommer un fichier simultanément

```
invite/> mv fichier.txt /home/utilisateur/Documents/nouveau_fichier.txt
```

- **Exemple :** Déplacer plusieurs fichiers vers un répertoire

```
invite/> mv fichier1.txt fichier2.txt /home/utilisateur/Documents/
```

Supprimer des fichiers avec `rm`

- La commande `rm` permet de supprimer des fichiers ou des répertoires

- **Exemple :** Supprimer un fichier

```
invite/> rm fichier.txt
```

- **Exemple :** Supprimer plusieurs fichiers

```
invite/> rm fichier1.txt fichier2.txt
```

- **Exemple :** Supprimer un répertoire et son contenu

```
invite/> rm -r repertoire/
```

- **Exemple :** Forcer la suppression sans confirmation

```
invite/> rm -f fichier.txt
```

- **Exemple :** Supprimer un fichier de manière interactive

```
invite/> rm -i fichier.txt
```

Créer des liens avec `ln`

- La commande `ln` permet de créer des liens entre des fichiers ou des répertoires

- Lien physique (hard link)

```
invite/> ln fichier1.txt lien1.txt
```

- Lien symbolique (soft link) :

```
invite/> ln -s /chemin/vers/fichier2.txt lien2.txt
```

- Créer un lien symbolique vers un répertoire :

```
invite/> ln -s /home/utilisateur/documents docs
```

- Supprimer un lien symbolique :

```
invite/> rm lien2.txt
```

Globbering - Partie 1

- **Problématique** : Comment exécuter une commande sur un sous-ensemble de fichiers ?
 - Comment afficher uniquement les images ?
 - Comment supprimer uniquement les fichiers textes ?
- **Définition** : Le **globbing** permet de sélectionner un ensemble de noms de fichiers/dossiers en utilisant des caractères génériques (*wildcards*).
- **Utilisation** : Le globbing peut être utilisé avec toutes les commandes Linux telles que **ls**, **rm**, **cat**, **find**, **head**, **tail**...
- **Syntaxe des Wildcards** :

Wildcard	Description	Exemple	Matches	Not Match
*	Match pour tous les nombres, caractères	Law*	Law, Laws où Lawyer	GrokLaw, La où aw
*	Match pour tous les nombres, caractères	*Law*	Law, GrokLaw où Lawyer	La où aw
?	Match un seul caractères	?at	Cat, cat, Bat où bat	at
[abc]	Match un seul caractère donné dans les brackets	[CB]at	Cat où Bat	cat où bat
[a-z]	Match un caractère de la plage indiqué dans les brackets	Letter[0-9]	Letter0, Letter1 jusqu'à Letter9	Letters, Letter où Letter10

Globbering - Partie 2

Quelques exemples en utilisant la commande `ls` et `rm` :

- Avec `ls` :

```
invite/> ls f*
invite/> ls fil*
invite/> ls *.jpg
invite/> ls *.txt
invite/> ls [cb]at*
```

- Avec `rm` :

```
invite/> rm -rf f*
invite/> rm -rf fil*
invite/> rm -rf *.jpg
invite/> rm -rf *.txt
invite/> rm -rf [cb]at*
```

- **Résumé :**

- Compréhension de l'organisation des fichiers et répertoires sous Windows et Linux
- Maîtrise des commandes de base pour gérer les répertoires et fichiers (`cd`, `mkdir`, `rmdir`, `pwd`)
- Introduction au globbing pour la gestion avancée des fichiers
- Comparaison entre les interfaces graphiques et les interfaces en ligne de commande

- **Prochaines Étapes :**

- Approfondir les commandes avancées et les options disponibles
- Explorer la création de scripts complexes pour automatiser des processus
- Comprendre les systèmes de gestion de fichiers en détail